

Python OOP + NumPy — Complete Learning Module

Part 1 — Object-Oriented Programming (OOP)

1. What is OOP?

Object-Oriented Programming is a way of structuring programs around **objects**.

Think of a real-world example:

A Car has **properties** and can **perform actions**.

Real-world concept	Python OOP
Car	Class
My BMW	Object
Color	Attribute
Speed	Attribute
Start	Method
Stop	Method

Class vs Object

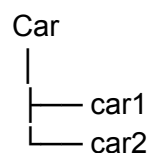
A **class** is a blueprint.

An **object** is an actual instance created from that blueprint.

```
class Car:  
    pass
```

```
car1 = Car()  
car2 = Car()
```

Here:



`Car` is the class.

`car1` and `car2` are objects.

2. Creating a Class

```
class Student:
```

```
    def display(self):  
        print("Student details")
```

Create an object:

```
student1 = Student()
```

```
student1.display()
```

Output:

Student details

What is `self`?

`self` represents the **current object**.

```
class Student:
```

```
    def display(self):  
        print(self)
```

When:

```
student1.display()
```

Python internally behaves approximately like:

```
Student.display(student1)
```

Therefore, `self` refers to `student1`.

3. Attributes

Attributes represent the **data/state** of an object.

```
class Student:
```

```
    def set_details(self, student_id, name, marks):
        self.student_id = student_id
        self.name = name
        self.marks = marks
```

Usage:

```
student1 = Student()
```

```
student1.set_details(101, "Rahul", 85)
```

```
print(student1.name)
print(student1.marks)
```

Output:

```
Rahul
85
```

4. Constructor — `__init__`

The constructor is automatically called when an object is created.

```
class Student:
```

```
    def __init__(self, student_id, name, marks):
        self.student_id = student_id
        self.name = name
        self.marks = marks
```

Now:

```
student1 = Student(101, "Rahul", 85)
student2 = Student(102, "Priya", 92)
```

Each object gets its own data.

```
print(student1.name)
print(student2.name)
```

Output:

```
Rahul
Priya
```

Why use a constructor?

Instead of:

```
student = Student()
student.set_details(101, "Rahul", 85)
```

we can simply write:

```
student = Student(101, "Rahul", 85)
```

5. Methods

Methods represent **behaviour**.

class Student:

```
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks
```

```
    def display(self):
        print(self.name, self.marks)
```

```
    def is_passed(self):
        return self.marks >= 40
```

Usage:

```
student = Student("Rahul", 75)
```

```
student.display()
```

```
print(student.is_passed())
```

Output:

Rahul 75

True

6. Instance Variables vs Class Variables

Instance variable

Each object has its own value.

```
class Student:

    def __init__(self, name):
        self.name = name
```

Class variable

Shared by all objects.

```
class Student:

    school = "ABC School"

    def __init__(self, name):
        self.name = name
```

Usage:

```
s1 = Student("Rahul")
s2 = Student("Priya")

print(s1.school)
print(s2.school)
```

Both objects access:

ABC School

7. Destructor — `__del__`

A destructor can be defined using `__del__`.

```
class Student:
```

```
    def __init__(self, name):
        self.name = name
        print("Object created")

    def __del__(self):
        print("Object destroyed")
```

Example:

```
student = Student("Rahul")
```

```
del student
```

Output may be:

```
Object created
Object destroyed
```

Important

Do **not** rely on `__del__` for critical resource cleanup. Python's garbage collection and interpreter shutdown can affect exactly when it runs.

8. Encapsulation

Encapsulation means **bundling data and methods together while controlling access to internal data**.

Python doesn't enforce private variables in the same way as some languages, but naming conventions and name mangling are used.

Public

```
class Employee:
```

```
    def __init__(self, salary):
        self.salary = salary
```

Accessible directly:

employee.salary

Protected convention

class Employee:

```
def __init__(self, salary):  
    self._salary = salary
```

`_salary` means:

"This is intended for internal/subclass use."

Private/name-mangled

class Employee:

```
def __init__(self, salary):  
    self.__salary = salary
```

Direct access:

employee.__salary

will generally fail.

Instead:

class Employee:

```
def __init__(self, salary):  
    self.__salary = salary
```

```
def get_salary(self):  
    return self.__salary
```

Then:

employee.get_salary()

9. Encapsulation with Validation

A practical example:

```
class BankAccount:
```

```
    def __init__(self, balance):
        self.__balance = balance

    def deposit(self, amount):

        if amount > 0:
            self.__balance += amount

    def withdraw(self, amount):

        if amount <= self.__balance:
            self.__balance -= amount
        else:
            print("Insufficient balance")

    def get_balance(self):
        return self.__balance
```

Usage:

```
account = BankAccount(10000)

account.deposit(5000)
account.withdraw(2000)

print(account.get_balance())
```

Output:

13000

The user cannot arbitrarily manipulate `__balance` through the normal public interface.

10. Inheritance

Inheritance allows one class to **reuse and extend another class**.

```
class Animal:
```

```
def eat(self):  
    print("Animal is eating")
```

```
class Dog(Animal):
```

```
    def bark(self):  
        print("Dog is barking")
```

Usage:

```
dog = Dog()
```

```
dog.eat()  
dog.bark()
```

Output:

```
Animal is eating  
Dog is barking
```

Here:

```
Animal  
  ↑  
  |  
Dog
```

Dog inherits from Animal.

11. Constructor with Inheritance

```
class Animal:
```

```
    def __init__(self, name):  
        self.name = name
```

```
class Dog(Animal):
```

```
    def __init__(self, name, breed):
```

```
super().__init__(name)
self.breed = breed
```

Usage:

```
dog = Dog("Bruno", "Labrador")
```

```
print(dog.name)
print(dog.breed)
```

Output:

```
Bruno
Labrador
```

super()

super() allows the child class to access functionality from its parent class.

12. Types of Inheritance

Single inheritance

A

↓

B

```
class A:
    pass
```

```
class B(A):
    pass
```

Multilevel inheritance

A

↓

B

↓

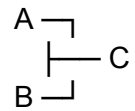
C

```
class A:
    pass
```

```
class B(A):  
    pass
```

```
class C(B):  
    pass
```

Multiple inheritance

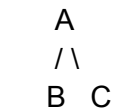


```
class A:  
    pass
```

```
class B:  
    pass
```

```
class C(A, B):  
    pass
```

Hierarchical inheritance



```
class A:  
    pass
```

```
class B(A):  
    pass
```

```
class C(A):  
    pass
```

13. Polymorphism

Polymorphism means:

Same interface, different behaviour.

Example:

```
class Dog:
```

```
def sound(self):  
    print("Bark")
```

```
class Cat:
```

```
    def sound(self):  
        print("Meow")
```

Now:

```
animals = [Dog(), Cat()]
```

```
for animal in animals:  
    animal.sound()
```

Output:

```
Bark  
Meow
```

The same:

```
animal.sound()
```

produces different behaviour depending on the object.

14. Method Overriding

```
class Animal:
```

```
    def sound(self):  
        print("Animal sound")
```

```
class Dog(Animal):
```

```
    def sound(self):  
        print("Bark")  
dog = Dog()
```

```
dog.sound()
```

Output:

Bark

The child class **overrides** the parent's method.

15. Abstraction

Abstraction means:

Show only what is necessary and hide implementation details.

For example, when using:

```
car.start()
```

you don't need to know every internal engine operation.

Python provides abstraction through the `abc` module.

```
from abc import ABC, abstractmethod
```

```
class Animal(ABC):
```

```
    @abstractmethod
    def sound(self):
        pass
```

Child class:

```
class Dog(Animal):
```

```
    def sound(self):
        print("Bark")
```

Now:

```
dog = Dog()
dog.sound()
```

Output:

Bark

But:

```
animal = Animal()
```

is not allowed because `Animal` contains an abstract method.

16. Magic Methods

Magic methods are special methods surrounded by double underscores.

Examples:

```
__init__  
__str__  
__repr__  
__len__  
__eq__  
__lt__  
__add__  
__del__
```

They allow Python objects to interact naturally with built-in operations.

`__str__`

Without `__str__`:

```
class Student:
```

```
    def __init__(self, name, marks):  
        self.name = name  
        self.marks = marks
```

Printing:

```
student = Student("Rahul", 90)
```

```
print(student)
```

doesn't produce a useful human-readable representation.

With `__str__`:

```
class Student:
```

```
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def __str__(self):
        return f'{self.name} - {self.marks}'
```

Now:

```
print(student)
```

Output:

Rahul - 90

17. `__repr__`

`__repr__` is generally intended to provide an **unambiguous/developer-oriented representation**.

```
class Student:
```

```
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def __repr__(self):
        return f'Student('{self.name}', {self.marks})'
student = Student("Rahul", 90)
```

```
print(repr(student))
```

Output:

Student('Rahul', 90)

18. `__len__`

class Team:

```
def __init__(self, players):  
    self.players = players
```

```
def __len__(self):  
    return len(self.players)
```

Usage:

```
team = Team(["A", "B", "C", "D"])
```

```
print(len(team))
```

Output:

4

19. `__eq__`

Allows objects to be compared using `==`.

class Student:

```
def __init__(self, student_id):  
    self.student_id = student_id
```

```
def __eq__(self, other):  
    return self.student_id == other.student_id
```

```
s1 = Student(101)
```

```
s2 = Student(101)
```

```
print(s1 == s2)
```

Output:

True

20. `__add__`

We can define what `+` means for our objects.

class Number:

```
def __init__(self, value):
    self.value = value

def __add__(self, other):
    return Number(self.value + other.value)

def __str__(self):
    return str(self.value)
```

Usage:

```
a = Number(10)
b = Number(20)
```

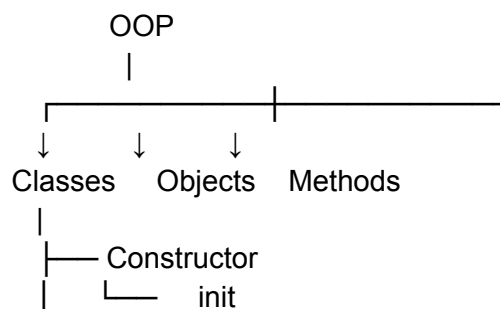
```
c = a + b
```

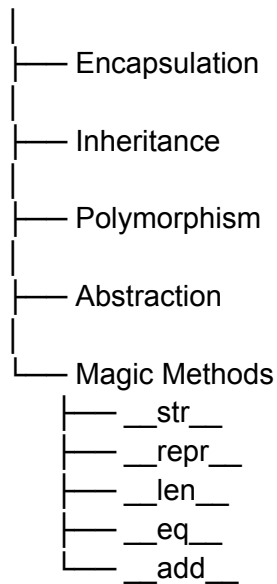
```
print(c)
```

Output:

```
30
```

OOP — Complete Mental Model





Part 2 — NumPy

21. What is NumPy?

NumPy = Numerical Python

It is one of the fundamental Python libraries for numerical computing.

Install:

```
pip install numpy
```

Import:

```
import numpy as np
```

NumPy's most important object is the:

```
numpy.ndarray
```

22. Python List vs NumPy Array

Python list:

```
numbers = [10, 20, 30, 40]
```

NumPy array:

```
numbers = np.array([10, 20, 30, 40])
```

NumPy allows efficient vectorized numerical operations.

```
numbers = np.array([10, 20, 30, 40])
```

```
print(numbers * 2)
```

Output:

```
[20 40 60 80]
```

Compare with a Python list:

```
numbers = [10, 20, 30, 40]
```

```
print(numbers * 2)
```

Output:

```
[10, 20, 30, 40, 10, 20, 30, 40]
```

This is a critical distinction for trainees.

23. Creating NumPy Arrays

From list

```
arr = np.array([10, 20, 30, 40])
```

From tuple

```
arr = np.array((10, 20, 30, 40))
```

2D array

```
arr = np.array([
    [10, 20, 30],
    [40, 50, 60]
])
```

24. Array Properties

```
arr = np.array([
    [10, 20, 30],
    [40, 50, 60]
])
```

ndim

Number of dimensions:

```
print(arr.ndim)
```

Output:

2

shape

Rows and columns:

```
print(arr.shape)
```

Output:

(2, 3)

size

Total number of elements:

```
print(arr.size)
```

Output:

6

dtype

Data type:

```
print(arr.dtype)
```

25. arange

Creates evenly spaced values within a range.

```
arr = np.arange(1, 11)
```

```
print(arr)
```

Output:

```
[ 1  2  3  4  5  6  7  8  9 10]
```

With step:

```
np.arange(0, 20, 2)
```

Output:

```
[ 0  2  4  6  8 10 12 14 16 18]
```

26. linspace

Creates a specified number of evenly spaced values.

```
np.linspace(0, 10, 5)
```

Output:

```
[ 0.  2.5  5.  7.5 10.]
```

Difference:

arange → specify the step

linspace → specify the number of values

27. Special Arrays

Zeros

```
np.zeros(5)
```

Ones

```
np.ones(5)
```

Full

```
np.full(5, 7)
```

Result:

```
[7 7 7 7 7]
```

Identity matrix

```
np.eye(3)
```

28. Indexing

```
arr = np.array([10, 20, 30, 40, 50])  
print(arr[0])
```

Output:

```
10  
print(arr[-1])
```

Output:

```
50
```

29. Slicing

```
arr[1:4]
```

Output:

```
[20 30 40]  
arr[:3]
```

Output:

```
[10 20 30]  
arr[::2]
```

Output:

```
[10 30 50]
```

30. 2D Indexing

```
arr = np.array([  
    [10, 20, 30],  
    [40, 50, 60],  
    [70, 80, 90]  
])
```

Get 50:

```
print(arr[1, 1])
```

Get second row:

```
print(arr[1])
```

Get third column:

```
print(arr[:, 2])
```

Output:

```
[30 60 90]
```

31. Vectorized Operations

This is one of the most important NumPy concepts.

```
sales = np.array([100, 200, 300, 400])
```

Add 10:

```
sales + 10
```

Multiply:

`sales * 2`

Divide:

`sales / 10`

Power:

`sales ** 2`

32. Mathematical Functions

`numbers = np.array([1, 4, 9, 16])`

Square root:

`np.sqrt(numbers)`

Exponential:

`np.exp(numbers)`

Log:

`np.log(numbers)`

Absolute value:

`np.abs([-10, 20, -30])`

33. Aggregations

Given:

`marks = np.array([70, 80, 90, 60, 85])`

Sum

`np.sum(marks)`

Mean

`np.mean(marks)`

Minimum

`np.min(marks)`

Maximum

`np.max(marks)`

Standard deviation

`np.std(marks)`

Median

`np.median(marks)`

34. Axis — Extremely Important

Consider:

```
marks = np.array([
    [80, 70, 90],
    [60, 75, 85],
    [90, 95, 88]
])
np.sum(marks)
```

Total:

733

axis=0

Aggregate **down the rows**, producing one value per column.

```
np.sum(marks, axis=0)
```

axis=1

Aggregate **across the columns**, producing one value per row.

```
np.sum(marks, axis=1)
```

Mental model:

axis=0 → ↓
axis=1 → →

35. Boolean Filtering

```
marks = np.array([45, 67, 32, 89, 91, 28])
```

Find marks greater than 50:

```
marks[marks > 50]
```

Output:

```
[67 89 91]
```

Multiple conditions:

```
marks[(marks >= 40) & (marks <= 80)]
```

Use:

& → AND

| → OR

~ → NOT

Do not use Python's **and** / **or** for elementwise NumPy array conditions.

36. **where**

```
marks = np.array([30, 60, 80, 45])  
np.where(marks >= 50, "Pass", "Fail")
```

Output:

```
['Fail' 'Pass' 'Pass' 'Fail']
```

Business use:

```
salary = np.array([30000, 50000, 70000, 25000])
```

```
category = np.where(
```

```
    salary >= 50000,  
    "High",  
    "Low"  
)
```

37. Reshaping

```
arr = np.arange(1, 13)
```

```
print(arr)
```

Convert into 3×4 :

```
arr.reshape(3, 4)
```

Result:

```
[[ 1  2  3  4]  
 [ 5  6  7  8]  
 [ 9 10 11 12]]
```

Important:

The number of elements must remain the same.

12 elements

$3 \times 4 = 12$

$2 \times 6 = 12$

$4 \times 3 = 12$

38. Flatten

Convert multidimensional array into one dimension:

```
arr.flatten()
```

Example:

```
arr = np.array([  
    [1, 2],
```

```
[3, 4]
])

print(arr.flatten())
```

Output:

```
[1 2 3 4]
```

39. Transpose

```
arr = np.array([
    [1, 2, 3],
    [4, 5, 6]
])
arr.T
```

Result:

```
[[1 4]
 [2 5]
 [3 6]]
```

Rows become columns.

40. Concatenation

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
np.concatenate([a, b])
```

Output:

```
[1 2 3 4 5 6]
```

41. Stack

Vertical stack

```
np.vstack([a, b])
```

Horizontal stack

```
np.hstack([a, b])
```

Understand the difference rather than memorizing it.

42. Sorting

```
numbers = np.array([50, 10, 40, 20, 30])
```

```
np.sort(numbers)
```

Output:

```
[10 20 30 40 50]
```

Get sorting indices:

```
np.argsort(numbers)
```

This is especially useful when you want to sort another array according to the order of one array.

43. Random Numbers

NumPy provides a random-number module.

```
np.random.default_rng()
```

Modern NumPy code should generally prefer the newer Generator API.

Example:

```
rng = np.random.default_rng(42)
```

```
numbers = rng.integers(1, 100, size=10)
```

```
print(numbers)
```

Random decimal values:

```
rng.random(5)
```

Normal distribution:

```
rng.normal(50, 10, 100)
```

44. NumPy Data Analysis Example

Consider employee salaries:

```
salaries = np.array([
    35000,
    42000,
    55000,
    62000,
    48000,
    75000,
    90000
])
```

Average:

```
print(np.mean(salaries))
```

Highest:

```
print(np.max(salaries))
```

Lowest:

```
print(np.min(salaries))
```

Employees earning more than ₹50,000:

```
print(salaries[salaries > 50000])
```

Count:

```
print(np.sum(salaries > 50000))
```

45. Broadcasting

Broadcasting allows NumPy to perform operations between arrays of compatible shapes.

```
prices = np.array([100, 200, 300])
```

Apply 18% tax:

```
prices * 1.18
```

Output:

```
[118. 236. 354.]
```

A scalar `1.18` is automatically applied to every element.

46. Practical NumPy Example — Student Marks

```
import numpy as np
```

```
marks = np.array([
    [80, 75, 90],
    [60, 70, 65],
    [95, 88, 92],
    [45, 50, 40]
])
```

Average marks for each student

```
student_avg = np.mean(marks, axis=1)
```

```
print(student_avg)
```

Average marks for each subject

```
subject_avg = np.mean(marks, axis=0)
```

```
print(subject_avg)
```

Students with average ≥ 60

```
student_avg[student_avg >= 60]
```

Highest mark

```
np.max(marks)
```

Lowest mark

```
np.min(marks)
```

OOP + NumPy Integration

This is a useful exercise because it combines both topics.

```
import numpy as np
```

```
class Student:
```

```
    def __init__(self, name, marks):  
        self.name = name  
        self.marks = np.array(marks)
```

```
    def average(self):  
        return np.mean(self.marks)
```

```
    def highest_mark(self):  
        return np.max(self.marks)
```

```
    def lowest_mark(self):  
        return np.min(self.marks)
```

```
    def is_passed(self):  
        return np.all(self.marks >= 40)
```

```
    def __str__(self):  
        return f"{self.name} - Average: {self.average():.2f}"
```

Usage:

```
student = Student(  
    "Rahul",  
    [80, 75, 90, 85]
```

)

```
print(student)
```

```
print("Highest:", student.highest_mark())
```

```
print("Lowest:", student.lowest_mark())
```

```
print("Passed:", student.is_passed())
```

Possible output:

Rahul - Average: 82.50

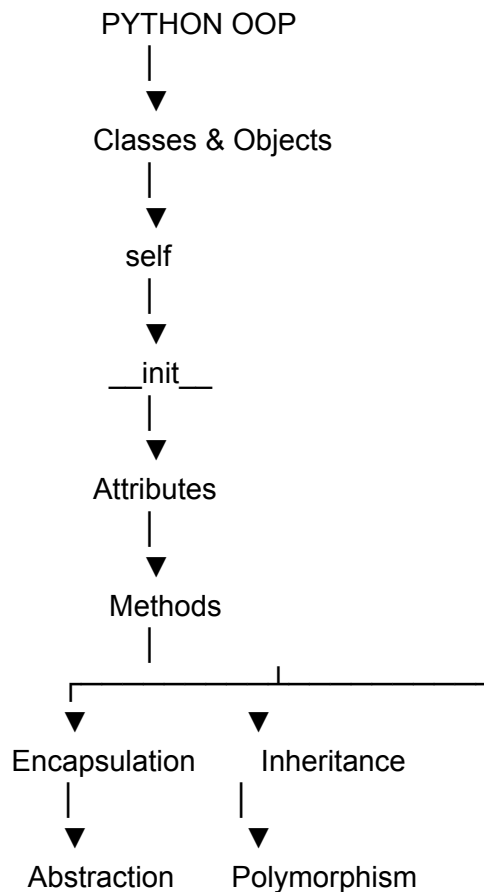
Highest: 90

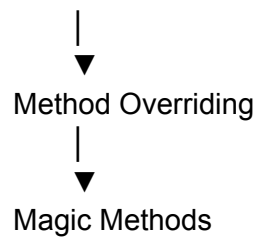
Lowest: 75

Passed: True

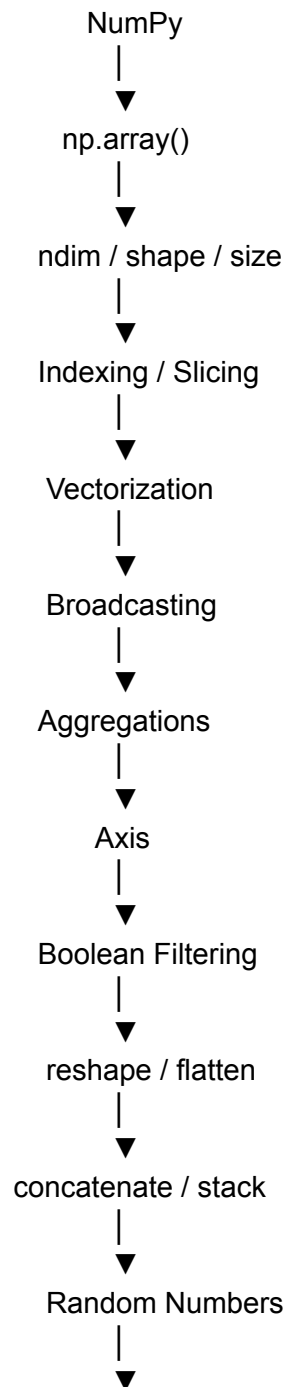
Recommended Training Flow

For trainees, I would teach this in this order:





Then move into:



Practice Progression

Level 1 — OOP Basics

1. Create a `Student` class.
2. Store name, ID and marks.
3. Create `display()` method.
4. Add `calculate_average()`.
5. Add `is_passed()`.

Level 2 — OOP Intermediate

6. Create `Employee` and `Manager` classes.
7. Demonstrate inheritance.
8. Override a method.
9. Demonstrate polymorphism.
10. Use encapsulation for salary.
11. Implement `__str__`.
12. Implement `__eq__`.

Level 3 — NumPy Basics

13. Create 1D and 2D arrays.
14. Practice indexing and slicing.
15. Calculate sum, mean, min and max.
16. Practice `axis=0` and `axis=1`.
17. Filter values using Boolean indexing.
18. Use `np.where()`.

Level 4 — NumPy Intermediate

19. Reshape arrays.
20. Flatten arrays.
21. Transpose matrices.
22. Concatenate arrays.
23. Stack arrays.
24. Sort arrays.
25. Use `argsort()`.
26. Generate random datasets.

Level 5 — Business Problems

27. Student marks analysis.
28. Employee salary analysis.
29. Monthly sales analysis.
30. Product price analysis.
31. Customer transaction analysis.
32. Create an OOP class that internally uses NumPy for numerical calculations.

Key takeaway for trainees:

OOP teaches you **how to structure and model a program**, while NumPy teaches you **how to efficiently work with numerical data**. Together, they form an important foundation before moving into **Pandas and data analysis**.